Spring Bean Lifecycle --- Complete Interview & Revision Notes

> **Source:** 30-page PDF --- *Spring Bean Lifecycle*
> **Goal:** Notebook writing + concept recall + interview preparation

---

Topics Covered in This PDF

This PDF covers:

1. What Spring Bean Lifecycle means
2. Complete Bean Lifecycle flow
3. Spring container startup
4. BeanDefinition
5. Sources of BeanDefinition
6. Bean instantiation
7. Dependency Injection during the lifecycle
8. Constructor, Setter and Field Injection
9. Aware interfaces
10. Initialization callbacks
11. `@PostConstruct`
12. `InitializingBean`
13. Custom `initMethod`
14. Bean ready-to-use phase
15. Destruction callbacks
16. `@PreDestroy`
17. `DisposableBean`
18. Custom `destroyMethod`
19. Singleton Bean Lifecycle
20. Prototype Bean Lifecycle

21. Prototype destruction behavior
22. Singleton depending on Prototype
23. `ObjectProvider` for fresh Prototype instances
24. `@Lazy` and lazy bean creation
25. Constructor circular dependency
26. Using `@PostConstruct` in a circular-dependency scenario
27. Final Bean Lifecycle revision and interview takeaways

---

1. What is Bean Lifecycle?

In normal Java, when we create an object ourselves:

```
PaymentService paymentService = new PaymentService();
```

we are responsible for the object's complete journey:

```
Create object
    ↓
Set required values
    ↓
Call required methods
    ↓
Clean up resources if required
```

In Spring, important application objects are normally managed by the **Spring IoC container**.

Example using `@Component`:

```
@Component
public class PaymentService {
}
```

Example using `@Bean`:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Once Spring manages the object, Spring takes responsibility for its lifecycle.

```
Spring discovers bean
        ↓
Creates object
        ↓
Injects dependencies
        ↓
Calls lifecycle callbacks
        ↓
Bean becomes ready
        ↓
Application uses bean
        ↓
Spring destroys bean when required
```

Simple Definition

Bean lifecycle means the complete journey of a Spring-managed object from the moment Spring discovers its definition until the moment Spring destroys it.

A Spring-managed object is called a **bean**.

Interview Definition

> Spring Bean Lifecycle is the process through which Spring discovers, creates, prepares, manages, and eventually destroys a Spring bean.

---

2. Complete Bean Lifecycle Flow

The PDF presents the lifecycle broadly as:

```
Spring Container Starts
        ↓
Reads Configuration / Annotations
        ↓
Creates BeanDefinition
        ↓
Instantiates Bean Object
        ↓
Injects Dependencies
        ↓
Calls Aware Interfaces
        ↓
Initialization Callbacks
        ↓
Bean is Ready to Use
        ↓
Application Uses Bean
        ↓
Destruction Callbacks
        ↓
Bean is Removed
```

The more detailed final revision flow also includes `BeanPostProcessor` stages:

```
BeanDefinition
      ↓
Object Creation
      ↓
Dependency Injection
      ↓
Aware Callbacks
      ↓
BeanPostProcessor — Before Initialization
      ↓
Initialization Callbacks
      ↓
BeanPostProcessor — After Initialization
      ↓
Bean Ready
      ↓
Application Uses Bean
      ↓
Destruction Callbacks
```

> **Must Remember:** Object creation and initialization are **not the same phase**.

---

3. Step 0 --- Spring Container Starts

Example:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.class);
```

We are not passing an object of `AppConfig`.

We are passing the **class metadata**.

Spring uses reflection to inspect the class.

It checks things such as:

- Does the class have `@Configuration`?
- Does it have `@ComponentScan`?
- Does it contain `@Bean` methods?
- Which package should be scanned?
- Which bean definitions should be registered?

So this is where Spring starts understanding the application structure.

```
AppConfig.class
      ↓
Spring inspects metadata
      ↓
@Configuration
@ComponentScan
@Bean methods
      ↓
Bean definitions discovered
```

---

4. Step 1 --- BeanDefinition is Read

Before creating objects, Spring first collects metadata about the beans.

This metadata is called a **BeanDefinition**.

A BeanDefinition can contain information such as:

```
Bean name
Bean class
Scope
Lazy / eager information
Dependencies
Initialization method
Destruction method
```

Example:

```
Bean name       → paymentService
Bean class      → PaymentService
Scope           → singleton
Lazy            → false
Dependencies    → ...
Init method     → ...
Destroy method  → ...
```

At this stage, the actual object is not necessarily created.

Spring is first registering information about the bean.

Why BeanDefinition First?

Spring is not creating only one object.

It is building an **object network**.

Example:

```
OrderService
      ↓ depends on
PaymentService
      ↓ depends on
PaymentGateway
      ↓ depends on
ApiClient
```

Spring first needs to understand:

- Which classes should be managed?
- What is the bean name?
- What is the scope?
- What dependencies are required?
- Is the bean lazy or eager?
- Does it have initialization logic?
- Does it have destruction logic?

Only after understanding this structure does Spring create objects.

---

5. Sources of BeanDefinition

The PDF shows three common sources.

5.1 Annotation-Based Configuration

```
@Configuration
@ComponentScan
public class AppConfig {
}
```

```
@Component
public class PaymentService {
}
```

Spring discovers the component and creates its BeanDefinition.

5.2 Java-Based `@Bean` Configuration

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

The `@Bean` method provides the bean definition.

5.3 XML-Based Configuration

```
<bean id="paymentService"
      class="in.strikes.PaymentService"/>
```

Spring can create the BeanDefinition from XML configuration as well.

Recall

```
BeanDefinition Sources

Annotations
    +
@Bean configuration
    +
XML configuration
    ↓
BeanDefinition
```

---

6. Step 2 --- Bean Object is Instantiated

After reading the BeanDefinition, Spring creates the actual object.

This is called **instantiation**.

Example:

```
@Component
public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService constructor called");
    }
}
```

When Spring creates the bean, the constructor is called.

Important Difference

Instantiation

Object is created.

Initialization

Object is prepared after creation.

Therefore:

```
Instantiation ≠ Initialization
```

Interview Question

Q: Are object creation and bean initialization the same?

Answer: No.

Instantiation means creating the object. Initialization is the later preparation phase where Spring performs initialization callbacks and related lifecycle processing.

---

7. Step 3 --- Dependencies are Injected

After or during object creation, Spring supplies required dependencies.

This is **Dependency Injection (DI)**.

The exact timing depends on the injection type.

---

8. Constructor Injection

Example:

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Flow:

```
Spring needs OrderService
        ↓
OrderService needs PaymentService
        ↓
Spring creates/finds PaymentService
        ↓
Spring passes PaymentService
into OrderService constructor
        ↓
OrderService object is created
```

The dependency is supplied **while the object is being created**.

Interview Point

Constructor injection is useful because required dependencies are available when the object is constructed.

---

9. Setter Injection

Example:

```
@Component
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Flow:

```
Object is created
      ↓
Spring calls setter
      ↓
Dependency is assigned
```

So the object exists before the dependency is injected through the setter.

---

10. Field Injection

Example:

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

Flow:

```
Object is created
      ↓
Spring uses reflection
      ↓
Dependency is injected directly into field
```

The PDF notes that field injection is common in examples, but **constructor injection is generally preferred for clean and testable code**.

---

11. Step 4 --- Aware Interfaces

Sometimes a bean needs information from the Spring container.

For example:

- What is my bean name?
- Which BeanFactory created me?
- Which ApplicationContext am I running inside?

Spring provides **Aware interfaces** for this purpose.

Aware interfaces are callback interfaces.

A callback means:

> Spring automatically calls a method at the appropriate lifecycle stage.

Common Aware Interfaces

The PDF specifically discusses:

```
BeanNameAware
ApplicationContextAware
```

Example:

```
@Component("myCustomBeanName")
public class MyBean
        implements BeanNameAware, ApplicationContextAware {

    public MyBean() {
        System.out.println("1. Constructor called");
    }

    @Override
    public void setBeanName(String name) {
        System.out.println("2. BeanNameAware called");
        System.out.println("Bean name: " + name);
    }

    @Override
    public void setApplicationContext(
            ApplicationContext applicationContext) {

        System.out.println("3. ApplicationContextAware called");
        System.out.println(
            "ApplicationContext: "
            + applicationContext.getClass().getSimpleName()
        );
    }
}
```

Possible output:

```
1. Constructor called
2. BeanNameAware called
Bean name: myCustomBeanName
3. ApplicationContextAware called
ApplicationContext: AnnotationConfigApplicationContext
```

Why is it called `setBeanName()`?

Because the direction is:

```
Spring Container
      ↓
Bean
```

Spring is **giving information to the bean**.

So Spring calls `setBeanName()` to set the bean name into the bean.

If we manually call:

```
setBeanName("abc");
```

it is just a normal Java method call.

It does **not** change the actual name registered inside the Spring container.

Do We Normally Use Aware Interfaces?

Usually, **no** for normal business logic.

They are more useful in:

- Framework-level code
- Logging utilities
- Custom libraries
- Infrastructure components

Interview Recall

```
Aware = Bean asks Spring for container-related information
```

---

12. Step 5 --- Initialization Callbacks

At this point:

```
Bean created
      ↓
Dependencies injected
      ↓
Aware callbacks completed
      ↓
Initialization
```

Initialization gives the bean a chance to execute startup logic before it is used.

Initialization logic can include:

- Validating configuration
- Initializing an internal cache
- Checking required setup
- Preparing resources
- Filling calculated values
- Logging startup state

The PDF covers three common approaches.

---

13. Initialization Option 1 --- `@PostConstruct`

Example:

```
import jakarta.annotation.PostConstruct;

@Component
public class PaymentService {

    @PostConstruct
    public void init() {
        System.out.println("Bean initialized");
    }
}
```

Meaning:

> After dependencies are injected, Spring calls this method once during initialization.

This is clean and commonly used.

For Spring 6 / Spring 7, the annotation is:

```
jakarta.annotation.PostConstruct
```

If using Spring Core/Spring Context without Spring Boot, the Jakarta annotation API may need to be added separately.

Example dependency shown in the PDF:

```
<dependency>
    <groupId>jakarta.annotation</groupId>
    <artifactId>jakarta.annotation-api</artifactId>
    <version>3.0.0</version>
</dependency>
```

Must Remember

```
@PostConstruct
    ↓
After dependency injection
    ↓
Initialization logic
```

---

14. Initialization Option 2 --- `InitializingBean`

Example:

```
import org.springframework.beans.factory.InitializingBean;

@Component
public class PaymentService implements InitializingBean {

    @Override
    public void afterPropertiesSet() {
        System.out.println("Bean initialized");
    }
}
```

Spring calls:

```
afterPropertiesSet()
```

during the initialization phase.

Disadvantage

The class becomes coupled to a Spring-specific interface.

---

15. Initialization Option 3 --- Custom `initMethod`

Example:

```
@Configuration
public class AppConfig {

    @Bean(initMethod = "start")
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Bean class:

```
public class PaymentService {

    public void start() {
        System.out.println("Bean initialized");
    }
}
```

Here, we choose our own method name:

```
start()
```

and tell Spring to invoke it after dependency injection.

Advantage

The class does not need to implement a Spring-specific lifecycle interface.

---

16. Initialization Callback Order

If all three approaches are used together, the PDF gives the usual order as:

```
@PostConstruct
      ↓
InitializingBean.afterPropertiesSet()
      ↓
custom initMethod
```

In real projects, we normally do **not** use all three together.

Choose one clear approach.

Interview Question

Q: What are common ways to perform bean initialization?

Answer:

1. `@PostConstruct`

2. `InitializingBean.afterPropertiesSet()`
3. Custom `initMethod`

---

17. Step 6 --- Bean is Ready to Use

After initialization is complete, the bean is fully ready.

```
Bean created
      ↓
Dependencies injected
      ↓
Initialization completed
      ↓
Bean ready
```

Now other beans can use it safely.

For singleton beans, Spring stores the object and reuses the same instance.

---

18. Step 7 --- Destruction Callbacks

When the Spring container shuts down, Spring can give beans a chance to clean up resources.

This is the **destruction phase**.

Cleanup can include:

- Closing database connections
- Stopping background threads
- Releasing file handles
- Clearing resources
- Flushing pending data

The PDF covers three approaches.

---

19. Destruction Option 1 --- `@PreDestroy`

Example:

```
import jakarta.annotation.PreDestroy;

@Component
public class PaymentService {

    @PreDestroy
    public void cleanup() {
        System.out.println("Cleaning up bean");
    }
}
```

Spring calls this method before destroying the bean.

---

20. Destruction Option 2 --- `DisposableBean`

Example:

```
import org.springframework.beans.factory.DisposableBean;

@Component
public class PaymentService implements DisposableBean {

    @Override
    public void destroy() {
        System.out.println("Cleaning up bean");
    }
}
```

This works, but couples the class to Spring.

21. Destruction Option 3 --- Custom `destroyMethod`

Example:

```
@Configuration
public class AppConfig {

    @Bean(destroyMethod = "stop")
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Bean:

```
public class PaymentService {

    public void stop() {
        System.out.println("Cleaning up bean");
    }
}
```

Spring invokes `stop()` during destruction.

22. Destruction Callback Order

If all three are used together, the PDF gives:

```
@PreDestroy
      ↓
DisposableBean.destroy()
      ↓
custom destroyMethod
```

Again, in real projects, we normally choose one clear approach.

---

23. Full Lifecycle Summary

The PDF's complete summary is:

```
1. Spring container starts
          ↓
2. Reads configuration and annotations
          ↓
3. Creates BeanDefinition
          ↓
4. Instantiates bean object
          ↓
5. Injects dependencies
          ↓
6. Calls Aware interfaces
          ↓
7. Initialization callbacks
   ├── @PostConstruct
   ├── InitializingBean
   └── custom initMethod
          ↓
8. Bean ready to use
          ↓
9. Application uses bean
          ↓
10. Destruction callbacks
   ├── @PreDestroy
   ├── DisposableBean
   └── custom destroyMethod
          ↓
11. Bean removed
```

More Detailed Interview Flow

```
BeanDefinition
      ↓
Instantiation
      ↓
Dependency Injection
      ↓
Aware Callbacks
      ↓
BeanPostProcessor
(before initialization)
      ↓
Initialization Callbacks
      ↓
BeanPostProcessor
(after initialization)
      ↓
Ready
      ↓
Destruction Callbacks
```

> **Important:** The final PDF revision explicitly includes `BeanPostProcessor` before and after initialization.

---

24. Singleton Bean Lifecycle

By default, Spring beans are **singleton scoped**.

Singleton means:

> Spring creates one object for that bean definition inside the Spring container and reuses it.

Lifecycle:

```
Creation
   ↓
Dependency Injection
   ↓
Initialization
   ↓
Usage
   ↓
Destruction
```

Example:

```
@Component
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("@PostConstruct called");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("@PreDestroy called");
    }
}
```

When context starts:

```
Constructor called
@PostConstruct called
```

When context closes:

```
@PreDestroy called
```

So:

```
Spring creates it
      ↓
Spring stores it
      ↓
Spring reuses it
      ↓
Spring destroys it
```

Must Remember

For singleton beans, Spring manages the **complete lifecycle**.

---

25. Prototype Bean Lifecycle

Prototype means:

> A new object is created every time the bean is requested from Spring.

Example:

```
@Component
@Scope("prototype")
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }
}
```

Requesting it twice:

```
PaymentService p1 =
        context.getBean(PaymentService.class);

PaymentService p2 =
        context.getBean(PaymentService.class);
```

Results in:

```
p1 != p2
```

They are different objects.

---

26. Prototype Lifecycle Flow

```
Container starts
      ↓
BeanDefinition is read
      ↓
No prototype object usually created at startup
      ↓
Client requests prototype bean
      ↓
Spring creates new object
      ↓
Dependencies injected
      ↓
Aware callbacks
      ↓
Initialization callbacks
      ↓
Spring hands object to client
      ↓
Spring does not normally track it for destruction
```

Singleton vs Prototype

Singleton

```
Spring creates
      ↓
Spring stores
      ↓
Spring reuses
      ↓
Spring destroys
```

Prototype

```
Spring creates
      ↓
Spring prepares
      ↓
Spring gives object to caller
      ↓
Caller is responsible for its later lifecycle
```

Important

Spring manages the beginning of the prototype lifecycle, but **not the complete ending**.

---

27. Prototype Destruction Does Not Happen Automatically

Example:

```
@Component
@Scope("prototype")
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("@PostConstruct called");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("@PreDestroy called");
    }
}
```

Main code:

```
AnnotationConfigApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.class);

PaymentService p1 =
        context.getBean(PaymentService.class);

context.close();
```

Expected lifecycle output:

```
Constructor called
@PostConstruct called
```

Usually, we do **not** see:

```
@PreDestroy called
```

Why?

For a singleton:

```
Spring creates object
      ↓
Stores/tracks object
      ↓
Context closes
      ↓
Spring destroys object
```

For a prototype:

```
Spring creates object
      ↓
Initializes object
      ↓
Hands object to caller
      ↓
Spring does not keep tracking it
```

If Spring stored references to every prototype forever, the objects could remain reachable and create memory-management problems.

Therefore, prototype destruction is not automatically managed like singleton destruction.

Important Difference

Garbage collection ≠ Spring destruction callback

Java may garbage-collect a prototype when there are no references to it, but that does not mean Spring automatically invokes its destruction callback.

---

28. Important Case --- Singleton Depends on Prototype

Suppose:

```
OrderService      → Singleton
PaymentSession    → Prototype
```

Example:

```
@Component
public class OrderService {

    private final PaymentSession paymentSession;

    public OrderService(PaymentSession paymentSession) {
        this.paymentSession = paymentSession;
    }
}
```

```
@Component
@Scope("prototype")
public class PaymentSession {

    public PaymentSession() {
        System.out.println("PaymentSession created");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("PaymentSession destroyed");
    }
}
```

Since `OrderService` is singleton, it is created only once.

During its creation:

```
Create OrderService
       ↓
Need PaymentSession
       ↓
Create one PaymentSession
       ↓
Inject PaymentSession
       ↓
Store PaymentSession reference inside OrderService
```

After that, the same `PaymentSession` instance remains inside `OrderService`.

Very Important

Prototype does **not** mean:

```
new object for every method call
```

It means:

```
new object whenever Spring is asked for the prototype bean
```

---

29. Fresh Prototype Instance with `ObjectProvider`

If a singleton needs a fresh prototype object whenever needed, use a mechanism such as:

- `ObjectProvider`
- `Provider`
- Lookup method injection
- `ApplicationContext.getBean()`

The PDF demonstrates `ObjectProvider`.

Example:

```
@Component
public class OrderService {

    private final ObjectProvider<PaymentSession>
            paymentSessionProvider;

    public OrderService(
            ObjectProvider<PaymentSession> paymentSessionProvider) {

        this.paymentSessionProvider = paymentSessionProvider;
    }

    public void createOrder() {

        PaymentSession session =
                paymentSessionProvider.getObject();

        System.out.println(session);
    }
}
```

Now:

```
createOrder()
      ↓
getObject()
      ↓
Request PaymentSession from Spring
      ↓
New prototype instance
```

If called again:

```
createOrder()
      ↓
getObject()
      ↓
Another prototype instance
```

Interview Question

Q: How can a singleton obtain a fresh prototype instance each time?

Answer: Use mechanisms such as `ObjectProvider`, `Provider`, lookup method injection, or `ApplicationContext.getBean()`.

---

30. `@Lazy` and Bean Lifecycle

By default, singleton beans are **eagerly created**.

That means when the Spring container starts, singleton beans are usually created immediately.

If a singleton is marked `@Lazy`:

```
@Component
@Lazy
public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService created");
    }
}
```

Spring reads the BeanDefinition at startup, but delays actual object creation.

Flow:

```
Container starts
      ↓
BeanDefinition registered
      ↓
Object NOT created yet
      ↓
Someone requests the bean
      ↓
Spring creates bean
      ↓
Dependencies + initialization
      ↓
Bean ready
```

Therefore:

```
BeanDefinition creation → startup
Actual object creation   → delayed
Dependency injection     → delayed
Initialization           → delayed
```

Must Remember

`@Lazy` does **not** mean Spring ignores the bean.

It means Spring delays its actual creation.

---

31. Constructor Circular Dependency

Consider:

```
@Component
public class A {

    private B b;

    public A(B b) {
        this.b = b;
    }
}
```

And:

```
@Component
public class B {

    private A a;

    public B(A a) {
        this.a = a;
    }
}
```

Dependency graph:

```
A
↓
needs B
↓
B
↓
needs A
↓
A
```

Spring cannot create `A` without `B`.

But Spring cannot create `B` without `A`.

Therefore:

```
A cannot be created
      ↕
B cannot be created
```

So constructor-based circular dependency fails.

---

32. Using `@PostConstruct` in a Circular-Dependency Scenario

The PDF shows a way to move one side of the relationship out of constructor creation.

Example:

```
@Component
public class A {

    private B b;

    public A(B b) {
        this.b = b;
    }

    @PostConstruct
    public void setB() {
        b.setA(this);
    }
}
```

And:

```
@Component
public class B {

    private A a;

    public void setA(A a) {
        this.a = a;
    }
}
```

The idea is:

```
Create A with B
      ↓
A has B
      ↓
Initialization phase
      ↓
@PostConstruct
      ↓
B receives A
```

This avoids requiring both objects to be completely available through constructors at the same time.

> **Interview Note:** The key concept shown by the PDF is that moving part of the dependency setup into a later lifecycle phase can avoid the constructor deadlock.

---

33. Important Bean Lifecycle Concepts to Remember

Instantiation vs Initialization

```
Instantiation
= object creation

Initialization
= preparation after object creation
```

Never treat them as identical.

BeanDefinition vs Bean

```
BeanDefinition
= metadata/instructions about the bean

Bean
= actual Spring-managed object
```

Singleton vs Prototype

```
Singleton
→ one instance per bean definition/container
→ Spring manages complete lifecycle

Prototype
→ new instance whenever requested from Spring
→ Spring does not automatically manage final destruction
```

`@Lazy`

```
Normal singleton
→ usually created during startup

@Lazy singleton
→ definition known at startup
→ object creation delayed
```

`@PostConstruct`

```
Dependencies injected
      ↓
@PostConstruct
      ↓
Initialization logic
```

`@PreDestroy`

```
Context shutdown
      ↓
@PreDestroy
      ↓
Bean destruction
```

For prototype beans, automatic destruction callbacks are not normally invoked by Spring.

---

34. Interview Questions and Answers

Q1. What is Spring Bean Lifecycle?

Answer: It is the complete journey of a Spring-managed object from BeanDefinition discovery through creation, dependency injection, initialization, usage, and destruction.

Q2. What is BeanDefinition?

Answer: BeanDefinition is metadata that tells Spring how a bean should be managed, including information such as bean name, class, scope, lazy behavior, dependencies, initialization, and destruction configuration.

Q3. Does Spring create the bean while reading the BeanDefinition?

Answer: Not necessarily. BeanDefinition registration is the metadata phase. Actual object creation happens later according to scope and initialization behavior.

Q4. What is the difference between instantiation and initialization?

Answer:

```
Instantiation = object creation
Initialization = preparing the created object
```

Q5. What are common initialization callbacks?

Answer:

```
@PostConstruct
InitializingBean.afterPropertiesSet()
custom initMethod
```

Q6. What are common destruction callbacks?

Answer:

```
@PreDestroy
DisposableBean.destroy()
custom destroyMethod
```

Q7. What is an Aware interface?

Answer: An Aware interface allows a bean to receive specific information from the Spring container, such as its bean name or ApplicationContext.

Examples:

```
BeanNameAware
ApplicationContextAware
```

Q8. Why is `setBeanName()` called instead of `getBeanName()`?

Answer: Because Spring is providing the bean name to the bean. The direction is:

```
Spring → Bean
```

Q9. What happens to singleton beans when the context closes?

Answer: Spring invokes their applicable destruction callbacks and destroys the managed singleton instances.

Q10. Does Spring automatically call `@PreDestroy` for prototype beans?

Answer: Normally, no. Spring creates and initializes prototype instances but does not track their complete destruction lifecycle.

Q11. Does prototype mean a new object for every method call?

Answer: No.

Prototype means a new instance when the prototype bean is requested from Spring.

Q12. What happens when a singleton directly receives a prototype through dependency injection?

Answer: The prototype is created when the singleton is created and that same prototype reference remains inside the singleton.

Q13. How can a singleton get a fresh prototype every time?

Answer: Use mechanisms such as:

```
ObjectProvider
Provider
Lookup method injection
ApplicationContext.getBean()
```

Q14. What does `@Lazy` do?

Answer: It delays actual creation of a lazy bean. Spring can register/read its BeanDefinition at startup while postponing object creation until the bean is needed.

Q15. Why does constructor circular dependency fail?

Answer: Because each object requires the other object to be available as a constructor argument before it can be created.

```
A → B → A
```

Quick Revision

1. Bean Lifecycle in One Flow

```
Container Starts
      ↓
Read Configuration
      ↓
BeanDefinition
      ↓
Instantiate
      ↓
Inject Dependencies
      ↓
Aware Callbacks
      ↓
Before Initialization Processing
      ↓
Initialization
      ↓
After Initialization Processing
      ↓
Bean Ready
      ↓
Use Bean
      ↓
Destroy
```

---

2. Initialization Recall

```
@PostConstruct
      ↓
InitializingBean.afterPropertiesSet()
      ↓
custom initMethod
```

Use one clear approach in normal application code.

3. Destruction Recall

```
@PreDestroy
      ↓
DisposableBean.destroy()
      ↓
custom destroyMethod
```

4. Singleton Recall

```
One instance for a bean definition
      ↓
Created
      ↓
Stored
      ↓
Reused
      ↓
Destroyed by Spring
```

5. Prototype Recall

```
Request prototype
      ↓
New instance
      ↓
Dependencies injected
      ↓
Initialized
      ↓
Given to caller
      ↓
Spring does not normally track destruction
```

6. Prototype Inside Singleton

```
Singleton created
      ↓
Prototype created once
      ↓
Prototype injected
      ↓
Same prototype reference remains
```

For a fresh prototype:

```
ObjectProvider.getObject()
```

7. `@Lazy` Recall

```
Startup
   ↓
BeanDefinition known
   ↓
Object creation delayed
   ↓
Bean requested
   ↓
Object created
   ↓
Initialized
```

8. Circular Dependency Recall

```
A needs B
B needs A
   ↓
Constructor injection
   ↓
Cannot create either
   ↓
Circular dependency
```

The PDF demonstrates moving dependency setup into a later lifecycle stage using `@PostConstruct`.

30-Second Interview Recall

If the interviewer asks **"Explain Spring Bean Lifecycle"**, remember this sequence:

> **BeanDefinition → Instantiation → Dependency Injection → Aware callbacks → BeanPostProcessor before initialization → Initialization callbacks → BeanPostProcessor after initialization → Ready → Destruction callbacks**

Then explain the important special cases:

```
Singleton
→ complete lifecycle managed by Spring

Prototype
→ created/initialized by Spring
→ destruction not automatically managed

@Lazy
→ definition registered
→ object creation delayed

@PostConstruct
→ initialization callback

@PreDestroy
→ destruction callback

Constructor Circular Dependency
→ A → B → A
→ cannot construct both through constructors
```

Must Remember

- **BeanDefinition is metadata; it is not the actual bean object.**
- **Instantiation and initialization are different phases.**
- **Constructor injection supplies dependencies while creating the object.**
- **Setter/field injection happen after object creation.**
- **Aware interfaces allow Spring to provide container-related information to a bean.**
- `@PostConstruct` **is an initialization callback.**
- `InitializingBean.afterPropertiesSet()` **is another initialization mechanism.**
- **A custom** `initMethod` **can perform initialization without making the class implement a Spring interface.**
- `@PreDestroy` **is a destruction callback.**

- Singleton beans have their complete lifecycle managed by Spring.
- Prototype beans are created and initialized by Spring, but their destruction is not automatically managed like singleton destruction.
- Prototype does not mean a new object on every method call.
- A prototype directly injected into a singleton is normally created once for that singleton instance.
- `ObjectProvider` can be used when a singleton needs a fresh prototype instance on demand.
- `@Lazy` delays actual bean creation.
- Constructor circular dependency occurs when A requires B and B requires A during construction.
- The PDF's final lifecycle flow includes BeanPostProcessor processing before and after initialization.

Final One-Line Takeaway

> Spring Bean Lifecycle is not just object creation; it is the complete process through which Spring discovers, creates, prepares, manages, uses, and destroys a Spring-managed bean.